



13-MAVZU · 42 TADAN · I BOB. HTML VA CSS

Jamoa bilan ishlash va loyiha qilish. Netlify va Vercel'ga deploy

Jamoaviy loyiha bosqichlari, rollarni taqsimlash, Git branch, merge va konfliktlar, GitHub'da hamkorlik va Pull Request; saytni Netlify va Vercel'ga bepul joylash.

2 soat · amaliy

Bu darsda nimani o'rganasiz

- Haqiqiy dasturchilar jamoada qanday ishlashini;
- Loyihani rejalashtirish va vazifalarni taqsimlashni;
- Git **branch** (tarmoq), **merge** va **konflikt** tushunchalarini;
- GitHub'da hamkor qo'shish va **Pull Request** orqali kodni birlashtirishni;
- Tayyor saytni **Netlify** va **Vercel**'ga joylashni (deploy).

Nima uchun jamoada ishlash muhim?

Instagram, Telegram, Click kabi ilovalarni bitta odam yaratmagan — ularda o'nlab, yuzlab dasturchilar birga ishlaydi. Dasturchining eng qadrli ko'nikmalaridan biri — **jamoada ishlay olish**: vazifani bo'lishish, boshqalarning kodini tushunish, kelishish.



O'xshatish: futbol jamoasi

Futbolda hamma to'p ortidan yugursa, jamoa yutqazadi. Har bir o'yinchining o'z o'rni bor: darvozabon, himoyachi, hujumchi. Murabbiy reja tuzadi. Dasturlash jamoasida ham shunday: kimdir header va menyuni, kimdir galereyani, kimdir formani yozadi — va oxirida hammasi bitta saytga birlashadi.

Jamoaviy loyiha bosqichlari

- 1 **G'oya va maqsad.** Sayt nima haqida? Kimlar uchun? Asosiy sahifalari qaysilar?
- 2 **Reja (vazifalar ro'yxati).** Saytni kichik vazifalarga bo'ling: "Header va menyu", "Bosh sahifa hero bo'limi", "Galereya sahifasi", "Aloqa formasi", "Telefon versiyasi"...
- 3 **Rollarni taqsimlash.** Har bir vazifaga mas'ul belgilanadi.
- 4 **Kelishuvlar.** Ranglar palitrasi, shriftlar, fayl nomlari, CSS class nomlash qoidalari — hamma bir xil yozishi uchun.
- 5 **Ishlab chiqish.** Har kim o'z qismini alohida **branch**da yozadi.
- 6 **Tekshirish (code review).** Jamoadoshlar bir-birining kodini ko'rib chiqadi.
- 7 **Birlashtirish va deploy.** Hammasi asosiy branch'ga qo'shiladi va internetga chiqariladi.

Vazifalarni boshqarish: Kanban doskasi

Jamoalar vazifalarni **Kanban** doskasida kuzatadi — 3 ustunli jadval:

 Qilinishi kerak (To Do)	 Jarayonda (In Progress)	 Tayyor (Done)
Galereya sahifasi — <i>Madina</i> Aloqa formasi — <i>Javohir</i>	Header va menyu — <i>Sardor</i>	Ranglar va shriftlar tanlandi — <i>hamma</i> Papka tuzilmasi — <i>Sardor</i>

Buning uchun **GitHub Projects** (repozitoriyada "Projects" yorlig'i) yoki Trello'dan foydalanish mumkin. Oddiy variant — sinf doskasi va stikerlar!

Git branch — parallel ish

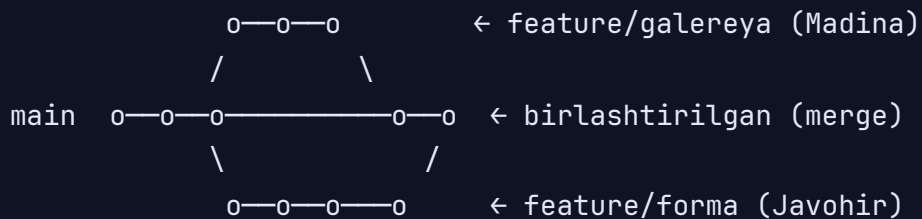
Branch (tarmoq) — loyihaning **alohida nusxasi**, unda asosiy kodga tegmasdan ishlash mumkin. Asosiy branch odatda `main` deb ataladi va unda doim **ishlaydigan** kod turadi.



O'xshatish: daraxt shoxlari

`main` — daraxtning tanasi. Har bir dasturchi o'z vazifasi uchun alohida shox (branch) o'zlashtiradi. Ish tugagach, shox tanaga qayta "payvand" qilinadi (**merge**). Shoxda xato bo'lsa — tana zarar ko'rmaydi.

Matn



Terminal

```
# Barcha branch'larni ko'rish (* - siz hozir qaysi branch'dasiz)
git branch

# Yangi branch yaratish va unga o'tish
git switch -c feature/galereya

# ... ishlash, git add, git commit ...

# Asosiy branch'ga qaytish
git switch main

# feature/galereya ni main ga birlashtirish
git merge feature/galereya
```

Konflikt — ikki kishi bitta qatorni o'zgartirsa

Madina va Javohir ikkalasi `index.html` dagi bitta sarlavhani turlicha o'zgartirishdi. Git qaysi birini tanlashni bilmaydi va **konflikt** haqida xabar beradi. Faylda shunday belgilar paydo bo'ladi:

Matn

```
<<<<<<< HEAD
<h1>Bizning maktab</h1>
=====
<h1>5-maktab rasmiy sayti</h1>
>>>>>>> feature/forma
```

- <<<<<<< HEAD va ===== orasida — sizning (joriy) variant;
- ===== va >>>>>>> orasida — birlashtirilayotgan branch varianti.

Hal qilish: jamoa bilan kelishib, keraklisini qoldiring, **barcha belgilarni o'chiring**, so'ng `git add .` va `git commit`. VS Code konfliktlarni rangli ko'rsatadi va "Accept Current / Incoming / Both" tugmalarini taklif qiladi.



Konfliktlarni kamaytirish

Har kim **turli fayllarda** ishlasin (masalan, har bir sahifa alohida fayl, har bir bo'lim CSS'i alohida). Tez-tez `git pull` qiling va kichik commit'lar bilan ishleng.

GitHub'da hamkorlik

1. Hamkor qo'shish

Jamoa sardori repozitoriyani yaratadi → **Settings** → **Collaborators** → **Add people** → jamoadoshlarning GitHub nomlarini qo'shadi. Ular emailga kelgan taklifni qabul qiladi va endi `git clone` qilib, `push` qila oladi.

2. Pull Request (PR) — kodni birlashtirishni so'rash

Professional jamoalarda hech kim to'g'ridan-to'g'ri `main` ga push qilmaydi. Buning o'rniga:

- 1 O'z branch'ingizda ishlaysiz va uni GitHub'ga yuborasiz: `git push -u origin feature/galereya`
- 2 GitHub'da sariq "**Compare & pull request**" tugmasi chiqadi — bosing.
- 3 Sarlavha va tavsif yozing: nima qildingiz, qanday tekshirish mumkin, skrinshot.

- 4 Jamoadoshlar **Files changed** yorlig'ida kodingizni ko'rib, izoh qoldiradi ("Bu rasmga alt qo'shish kerak").
- 5 Izohlarni tuzatib, yana push qilasiz — PR avtomatik yangilanadi.
- 6 Hamma rozi bo'lsa — **Merge pull request**. Kod `main` ga qo'shildi!
- 7 Hamma o'z kompyuterida yangilanadi: `git switch main` → `git pull`.

Code review madaniyati

- Kodni tanqid qiling, odamni emas: "Bu yerda `alt` yo'q" ✓, "Sen hech narsa bilmaysan" ✗.
- Maqtashni ham unutmang: "Animatsiya zo'r chiqibdi!" 👍
- Taklif sifatida yozing: "Bu yerda Grid ishlatsak qanday bo'ladi?"

Deploy — saytni internetga chiqarish

Deploy — tayyor loyihani foydalanuvchilar ochishi uchun serverga joylash. 11-mavzuda GitHub Pages'ni ko'rdik. Endi ikkita mashhur platforma:

	Netlify	Vercel
Narxi	Shaxsiy loyihalar uchun bepul	Shaxsiy loyihalar uchun bepul
Manzil	<code>nom.netlify.app</code>	<code>nom.vercel.app</code>
HTTPS	Avtomatik	Avtomatik
GitHub bilan ulanish	Ha — har push'da avtomatik yangilanadi	Ha — har push'da avtomatik yangilanadi
O'ziga xosligi	Papkani sudrab tashlab deploy (Drag & Drop), formalarni qabul qilish	React/Next.js loyihalari uchun ayniqsa qulay

Netlify — 1-usul: sudrab tashlash (eng oson)

- 1 `app.netlify.com/drop` sahifasini oching.

- 2 Sayt papkangizni (ichida `index.html` bor) sahifadagi maydonga sichqoncha bilan **sudrab tashlang**.
- 3 Bir necha soniyada sayt tayyor: `https://tasodifiy-nom-123.netlify.app`. Saqlab qolish uchun ro'yxatdan o'ting.
- 4 **Site configuration** → **Change site name** orqali chiroyli nom bering: `maktab5-samarqand.netlify.app`.

Netlify yoki Vercel — 2-usul: GitHub orqali (professional)

- 1 Netlify yoki Vercel'ga **GitHub akkauntingiz bilan** kiring.
- 2 **Add new site** → **Import an existing project** (Netlify) yoki **Add New** → **Project** (Vercel).
- 3 GitHub'dagi repozitoriyangizni tanlang.
- 4 Oddiy HTML/CSS sayt uchun hech qanday sozlama kerak emas (Build command — bo'sh, Publish directory — `/` yoki loyiha ildizi). **Deploy** bosing.
- 5 Endi har safar `git push` qilganingizda sayt **avtomatik yangilanadi**. Bu — **CI/CD** (uzluksiz yetkazib berish) degan professional usulning eng oddiy ko'rinishi.



O'z domeningiz

Agar `.uz` domen sotib olsangiz, Netlify/Vercel'da **Domain settings** → **Add custom domain** qilib, domen sozlamalarida (DNS, 1-mavzu) ular ko'rsatgan yozuvlarni kiritasiz.

Amaliy mashg'ulot: jamoaviy loyiha (8-loyiha)

3–4 kishilik jamoalarda **"Bizning sinf"** saytini yarating.

- 1 **Sardor** (jamoar rahbari) GitHub'da `bizning-sinf` repozitoriyasini yaratadi, boshlang'ich tuzilmani push qiladi va hamkorlarni qo'shadi:

Matn

```
bizning-sinf/
├── index.html           ← bosh sahifa (Sardor)
├── oquvchilar.html     ← o'quvchilar kartalari (2-a'zo)
├── galereya.html        ← rasmlar galereyasi (3-a'zo)
├── aloqa.html           ← forma va xarita (4-a'zo)
├── css/
│   ├── umumiy.css      ← ranglar, shriftlar, header, footer (kelishib
│   │                   yoziladi)
│   └── ...              ← har kimning sahifasi uchun alohida fayl
└── rasmlar/
```

- 2 Jamoa **kelishuvlar** faylini (`README.md`) yozadi: ranglar (HEX), shriftlar, class nomlash qoidasi, kim nimaga mas'ul.
- 3 Har bir a'zo `git clone` qiladi va o'z branch'ini yaratadi: `git switch -c feature/galereya`.
- 4 O'z sahifasini yozadi, kamida 3 ta commit qiladi va `git push -u origin feature/galereya`.
- 5 GitHub'da Pull Request ochadi; boshqa a'zo uni ko'rib, kamida bitta izoh qoldiradi; so'ng merge qilinadi.
- 6 Barcha PR'lar birlashgach, sardor loyihani Netlify yoki Vercel'ga GitHub orqali ulaydi.
- 7 Har bir a'zo telefonida saytni ochib tekshiradi; topilgan xatolar uchun GitHub'da **Issue** ochiladi va tuzatiladi.
- 8 **Taqdimot:** jamoa saytini sinfga 3 daqiqada tanishtiradi: kim nima qildi, qanday qiyinchiliklar bo'ldi.

Ko'p uchraydigan xatolar

- ✗ Hamma `main` ga to'g'ridan-to'g'ri push qiladi — bir-birining ishini o'chirib yuboradi.
- ✗ Ishni boshlashdan oldin `git pull` qilmaslik — eski kod ustida ishlab, keyin katta konflikt chiqadi.
- ✗ Konflikt belgilarini (`<<<<<<`) faylda qoldirib yuborish — sahifada g'alati matn ko'rinadi.
- ✗ Netlify'da "Page not found" — papka ichida yana papka (`sayt/sayt/index.html`); `index.html` yuklangan papkaning **ildizida** bo'lishi kerak.

O'zingizni tekshiring

? Git'da asosiy koddan alohida, mustaqil ishlash uchun nima yaratiladi?

- A. Commit
- B. Branch
- C. Clone
- D. Remote

? GitHub'da o'z branch'ingizni asosiy kodga qo'shishni so'rash nima deyiladi?

- A. Push Request
- B. Merge Commit
- C. Issue
- D. Pull Request

? Konflikt qachon yuzaga keladi?

- A. Ikki branch'da bitta faylning bitta qatori turlicha o'zgartirilganda
- B. Internet uzilganda
- C. Fayl o'chirilganda
- D. Juda ko'p commit qilinganda

? Netlify'ga GitHub orqali ulangan saytni yangilash uchun nima qilish kerak?

- A. Har safar papkani qayta yuklash
- B. Netlify'ga xat yozish
- C. Shunchaki `git push` qilish — sayt avtomatik yangilanadi
- D. Saytni o'chirib, qaytadan yaratish

? Kanban doskasining odatiy ustunlari qaysilar?

- A. HTML, CSS, JS
- B. Qilinishi kerak, Jarayonda, Tayyor
- C. Dushanba, Seshanba, Chorshanba
- D. Oson, O'rta, Qiyin



Uyga vazifa

1. Jamoaviy loyihadagi o'z qismingizni yakunlang va PR orqali birlashtiring.
2. Shaxsiy portfoliingizni Netlify'ga (GitHub orqali) ham joylang va manzilini GitHub profilingizga qo'shing.
3. Biror sinfdoshingizning repozitoriyasida "Issue" oching: saytida topgan bitta xato yoki taklifingizni hurmat bilan yozing.



Yodda tuting

- Jamoaviy ish: reja → rollar → kelishuvlar → branch'larda ishlash → code review → merge → deploy.
- `git switch -c nom` — yangi branch; `git merge nom` — birlashtirish; konfliktida belgilarni o'chirib, add + commit.
- GitHub: Collaborators, Pull Request, Issues, Projects.

- Deploy: Netlify (Drop yoki GitHub), Vercel (GitHub) — bepul, HTTPS, har push'da avtomatik yangilanish.

☐ Darsni o'qib bo'ldim

Test javoblari

1. Git'da asosiy koddan alohida, mustaqil ishlash uchun nima yaratiladi? — **B) Branch**
2. GitHub'da o'z branch'ingizni asosiy kodga qo'shishni so'rash nima deyiladi? — **D) Pull Request**
3. Konflikt qachon yuzaga keladi? — **A) Ikki branch'da bitta faylning bitta qatori turlicha o'zgartirilganda**
4. Netlify'ga GitHub orqali ulangan saytni yangilash uchun nima qilish kerak? — **C) Shunchaki git push qilish — sayt avtomatik yangilanadi**
5. Kanban doskasining odatiy ustunlari qaysilar? — **B) Qilinishi kerak, Jarayonda, Tayyor**